

PROGRAMMING ASSIGNMENT 1

SENSORS READINGS

COP3503 Computer Science 2

Spring 2026

Due Date

The assignment is due on **Sunday, February 1 at 11:59 PM EST** via Webcourses. **Do not email the professor or TAs. Submissions by email will not be accepted.**

Assignments may be submitted up to 24 hours late with a penalty. Please refer to the syllabus for details. To receive full credit, submit on time and allow extra time for uploading. Last minute submissions often fail due to slow Internet, resulting in late penalties.

The **Webcourses timestamp** determines submission time. If your submission is even one second past the deadline, it will be marked late. Internet issues are not an excuse. Please plan accordingly.

Assignment Description

This assignment will serve as a warmup that uses skills you have learned previously in your COP3502 course. In this assignment, you are given a sensor that records integer readings throughout an experiment. You will analyze these readings by answering queries of the form:

"How many sensor readings fall within the inclusive range [L, R]?"

In each query, L represents the lower bound of the range and R represents the upper bound of the range. A sensor reading is counted for a query if its value is greater than or equal to L and less than or equal to R. Each query is independent and may use different values of L and R.

Example: Suppose the sensor recorded the following readings:

[7, 1, 9, 2, 2, 10, 5]

Consider the query L = 2 and R = 7. This query asks: How many sensor readings are between 2 and 7, inclusive? The readings that satisfy this condition are:

7, 2, 2, 5

Since there are 4 such readings, the answer to this query is 4.

Your task will write two ways to solve this problem

- A Brute Force solution that runs within $O(nq)$.

- An Optimized solution that runs within $(O(n \log n + q \log n))$.

where, n represents the number of sensor readings in the array and q represents the number of range queries being processed.

The Provided Driver File

You have been provided with a driver file called `SensorRangeDriver.java`. This driver file is responsible for running a set of predefined test cases for the methods you will implement in `SensorRange.java`.

The driver file is structured in a manner similar to the Eustis assignment you have seen previously. The main method contains a switch statement, where each case corresponds to a specific test scenario.

Each test case generates a fixed set of sensor readings and range queries and then invokes both the brute force and optimized methods you are required to implement. The driver prints the sensor readings, the queries, and the results produced by your methods. These test cases are deterministic, meaning all students run their code on the same inputs.

Please do not modify the provided driver file in any way. This includes adding or removing any content. Changes to the driver file can prevent your code from being evaluated correctly, which may result in lost points that cannot be recovered. In some cases, modifications may cause your code to fail compilation, which will have a serious impact on your grade and will not be reassessed. Graders will use a driver file with modified test cases but will rely on the same setup provided in this assignment.

Input

The input to your methods consists of two parameters provided by the driver file:

- An integer array `readings[]` of size n , where each element represents a sensor reading.
- A two-dimensional integer array `queries[][]` of size $q \times 2$, where each row represents a range query. For a given query `queries[i]`, the values `queries[i][0]` and `queries[i][1]` correspond to the lower bound L and upper bound R of the range, respectively.

All input arrays are generated by the provided driver file. You should not generate or modify input data inside your methods.

Output

Each method returns an integer array of size q , where the value at index i represents the number of sensor readings that fall within the inclusive range $[L, R]$ for the i -th query.

If a query has $L > R$, the result for that query should be 0. Sensor readings may contain negative values and duplicates. Negative values should be treated like any other integer, and each occurrence of a duplicate value must be counted separately if it falls within the query range. The driver file will print the sensor readings, the queries, and the output returned by your methods. During grading, additional test cases will be used to verify correctness and performance.

Implementation Directions

Your file must contain this public class and these methods exactly:

```
1 public class SensorRange
2 {
3     public static int[] rangeCountBF(int[] readings, int[][] queries);
4
5     public static int[] rangeCountFast(int[] readings, int[][] queries);
6 }
```

Both methods take the same input parameters:

- `readings[]`: an integer array of size n containing sensor readings.
- `queries[][]`: a $q \times 2$ integer array where each row represents a query range (L, R) .

Each method must return an integer array of size q , where the value at index i represents the number of sensor readings that fall within the inclusive range $[L, R]$ for the i -th query.

The `rangeCountBF` method must use a brute-force approach by scanning the entire readings array for each query. The `rangeCountFast` method must use a more efficient approach based on sorting and binary search.

Note: Besides those two methods, you are welcome to add any other methods as long as you do not modify the driver script

Do not create a `main` method in your `SensorRange.java` file. You will use the `main` method provided in the driver file. Adding a `main` method to your file can cause your code to run incorrectly, resulting in point deductions that cannot be recovered.

Code Style

Students are expected to follow the code style guidelines provided for this course. The style guide is available on the assignment page, and students must adhere to it with this programming assignment. Code style will be enforced, and any violations may result in point deductions according to the rubric.

Testing my Code in Eustis

Make sure to test your code on Eustis before submitting your assignment, as this is the environment the TAs will use to compile and run your code. Failure to successfully compile or run on Eustis will result in a score of **zero**, even if the code runs correctly on your local machine.

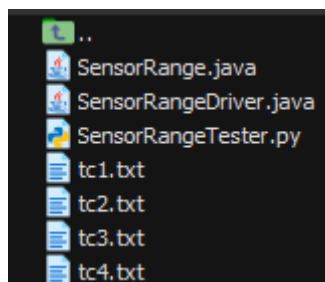
Directory Setup in Eustis

Once you have logged into Eustis, you will need to set up the directory properly in order to test your code. It is recommended that students create a separate folder for each assignment to avoid potential logical errors between different programming assignments.

After creating the directory for this assignment, upload the following files:

- `SensorRange.java`
- `SensorRangeDriver.java`
- `SensorRangeTester.py`
- `tc1.txt`
- `tc2.txt`
- `tc3.txt`
- `tc4.txt`

The following figure shows a visual of the directory setup with required files uploaded.



Manual Run through javac and java

After moving your files into the same directory on Eustis, make sure they are compiled before testing your assignment. To compile your code, use the following command:

```
javac SensorRangeDriver.java
```

If no messages are displayed, your code has compiled successfully with no errors or warnings. After compiling, you can execute the program by selecting the test case you wish to run. Use the following command format:

```
java SensorRangeDriver <test_case_number>
```

For example, to run test case 2, use:

```
java SensorRangeDriver 2
```

Note: Test case 5 is provided solely to demonstrate the runtime difference between the brute force solution and the optimized solution. You should not expect to obtain the same timing results, as execution time may vary across machines and runs. Your code will **not** be graded or tested using this test case.

How graders will test your code through python3

Once you have completed manual testing of all test cases individually and verified that your output is correctly formatted, you may perform a final test using the Python script provided for grading purposes. This script will execute all test cases and generate a report indicating whether all tests passed or if any cases are still incorrect.

The command to run the Python script is:

```
python3 SensorRangeTester.py
```

After running the script, you will receive a positive message if all test cases pass. You should notice that the script will pull your name directly from the file header comments (*please see the section **comment header** for more information*). If any test cases fail, a different message will appear.

Comment Header

Please make sure to provide the appropriate comment header as the very first content at the top of your file (including before any import statements). If your comment header is incorrect or not placed literally at the top of the file, points will be deducted according to the rubric.

The header must be formatted exactly as shown below and placed on the first line of the file with no whitespace above it. For example, if your name is Steve Harrington, the comment header should look like:

```
1 /* Steve Harrington
2    Sensor Range
3    COP3503 Computer Science 2
4    SensorRange.java
5 */
```

The format is as follows:

- First line: Your first and last name.
- Second line: The assignment name.

- Third line: The course code and course name.
- Fourth line: The name of the Java file.

This format must be followed exactly for the Python script to properly capture your name. Failure to follow this format will result in point deductions. The main reason for this requirement is that graders will use a special Python script to record your name during test runs.

Grading Rubric

Please refer to Webcourses for the official grading rubric. It is located on the assignment page.

A Very Important Note for Running Eustis and Packages! Read Carefully!

Many of you are probably using IDEs like Netbeans and Eclipse to build your Java Solutions. Please note that some of these IDEs will automatically place your Java file in some sort of package. Please make sure your Java file is not defined in some package as this can result package private errors. Any such error that occurs during the grading will not be fixed and points will be deducted as such in accordance with the respective categories in the rubric. Also, DO NOT create a main method in your solution file!! This will result in your code not running properly with the driver file which will result in points being deducted from the respective categories.

What to submit to Webcourse?

For this assignment, please submit your `SensorRange.java` file only.

Do not submit the driver file. The graders will use their own driver file with slightly modified test cases to evaluate your submission.

What happens if my submitted code doesn't compile or is stuck in an infinite loop? How will my code be graded?

If your code does not compile at all, the graders will not award any credit (a score of 0) for categories related to compilation or any functionality that depends on successful compilation.

If your code compiles but enters an infinite loop or throws an exception, points will be deducted in the categories directly affected by these issues.

Comments

For this assignment, you must include at least one comment in each method that is implemented in your file. It should include how the method is working. Please remember to follow the code style guidelines regarding how comments should be written.

Testing on Eustis

It is your responsibility to test your code on Eustis. If you submit your assignment without testing on Eustis, you risk losing points for issues that may behave differently on your operating system.

Remember: you cannot dispute grades if your code fails on Eustis, even if it worked on your machine. The Eustis environment is the final authority. Plan ahead and make sure to test on Eustis!

AI Policy

Use of Generative Artificial Intelligence (GenAI) is prohibited. Use of GenAI tools via website, app, or any other access, is not permitted in this class. All components of assignments in this course must be independently and originally completed by the student. Representing work created by GenAI as your own will be treated as plagiarism.